

SPINACH: Inferring Properties of Web Applications for Property-Based Testing

Anonymized

Abstract

Turning AI-generated web application prototypes into reliable, deployable software requires testing, yet writing property-based tests (PBTs) is difficult: developers struggle to identify meaningful properties, and LLMs asked to generate tests from natural language documentation often miss relevant ones. We present SPINACH, a VSCode extension that addresses this gap via a specification inference pipeline. An LLM agent first explores a web application and represents its functionality as *software concepts*, a user-facing behavioral specification formalism; a second agent derives natural language properties from these concepts for developer review; a final phase translates approved specifications into LTL and executable PBTs using the Bombadil framework. We ask whether high-level conceptual specifications improve LLM-generated PBT quality (RQ1) and whether they help developers extend and maintain AI-generated systems (RQ2). We report preliminary results applying SPINACH to two open-source applications, RealWorld and 4ga Boards, and outline planned evaluations using mutation testing and a user study of human-AI collaborative development.

ACM Reference Format:

Anonymized. 2026. SPINACH: Inferring Properties of Web Applications for Property-Based Testing. In . ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Vibe-coding tools like Lovable and Base44 [2, 4] make it possible to generate entire web applications with a few prompts. Turning these prototypes into deployable websites requires the developer to have reasonable assurance that a site visitor will not encounter a bug. With large, dynamic web applications, such assurances are typically gained through testing.

Property-based testing is an effective way of covering a large input space with a few tests. With PBT, it's possible

for developers to write tests that are able to exercise the many possible states a web application can be in. Recent work has shown PBTs to be around 50 times more effective at finding bugs than example-based tests [15], underscoring the potential of PBTs to reduce the burden of testing.

Yet, writing PBTs is an onerous process in its own way. Users often have difficulty coming up with properties to test with PBT and developing precise generators of inputs [7]. In 2026, LLMs are an obvious solution, and indeed these models are able to generate property-based tests that can find bugs for individual Python functions [10]. However, LLMs may not identify and test all relevant properties when given natural language documentation [16].

To address the issues of property identification when generating property-based tests with LLMs, we have developed a specification inference pipeline for web applications. We implement this pipeline in a VSCode extension called SPINACH. Given an existing web application, our tool first uses an agent that explores the app and represents the application's functionality in terms of software concepts [9]. Software concepts serve as a high-level behavioral specification language for web applications in our tool. Then, an additional agent writes natural language specifications based on these concepts, which the user can then review and mark as matching their intentions. The final step converts the English specifications into executable tests using the property-based testing framework Bombadil [3].

Unlike other methods of representing application structure, such as entity-relationship diagrams, software concepts are concerned with software behavior and are user-facing. Being user-facing, a conceptual analysis of a software system can encode a user's expectations of how the abstracted system components should work in relation to each other. For example, a social media app can have the concepts of *post*, *like*, and *comment* which can be implemented in a myriad of ways, but to the user, these concepts have clear-cut functionalities and relationships to each other. LLMs, having been exposed to countless websites during training, can simulate a naive user's unstated expectations [14]. Thus, LLMs may be able to conduct reasonable conceptual analyses of web applications.

We have two main research questions:

RQ1: Does having access to high-level specifications improve the quality of LLM-generated property-based tests?

RQ2: Do high-level specifications and property-based tests help developers construct systems that meet their needs when using AI tools?

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Due to our focus on web applications in this work, we use conceptual analyses as our high-level specification language and Bombadil's LTL-based language as the low-level executable language.

We plan to evaluate SPINACH by assessing the bug finding capability of the PBTs it produces. To evaluate the efficacy of using a conceptual analysis to infer specifications, we will seed bugs in web application source code via mutation testing [15]. Then, one agent will generate PBTs after exploring the application, and the other will generate the tests using only the conceptual model produced in the previous step in the pipeline. We will then compare the percentage of mutations caught by each set of PBTs. We can use test quality as a proxy for specification quality, but other methods such as symbolic testing of specifications [12] could be useful for this use case.

We also plan to evaluate how SPINACH can aid human-AI collaboration in software development. Having understandable high-level specifications could help developers add new features and maintain applications more easily. The derived executable properties can ensure that the high-level specs are being adhered to as the application changes. We plan to conduct a user study which provides participants with an existing web application and PBT test suite. Their task is to add a given feature to this application without creating breaking changes. Participants will be randomly assigned to a group that either has access to a high-level conceptual specification or does not. Then, each completed implementation will be evaluated on a reference test suite.

2 System Design

We have designed a three phase system for generating PBTs for web applications and outline the pipeline in Figure 3. First, an AI agent crawls the web application under test and proposes a high-level specification that the developer can refine. This includes natural language descriptions of *concepts*, abstract units of functionality that compose applications [9], and how they relate to each other. Users carry tacit expectations of website behavior, and concepts can be used to articulate these expectations [6]. The LLM agent crawls the existing site and outputs a conceptual analysis [9] that models the concepts as state machines and uses *synchronizations* to tie the actions of different concepts together to compose them. Figure 1 is an example of a concept description from a task management application generated by SPINACH. This concept describes a user and notes its purpose, operational principle, state fields, actions, and invariants that must hold. The model also outputs integrity notes, highlighting where this concept may be too far-reaching, but points out that this design choice is standard practice.

Effective UI exploration is still an active area of research,

In the second phase, the agent derives specific natural language properties based on these concepts, which the user can also refine. For example, the tool generated the natural

```

### User
Purpose: Represents a person who owns and
        collaborates on projects, boards, and
        cards.
Operational principle: Registering (or admin
        creation) creates a User with a
        username and email. The user logs in,
        updates their profile, and appears as a
        member on cards and boards.
State:
- name: display name
- username: unique login handle
- email: unique address
- role: admin | regular
- createdAt: timestamp
Actions:
- adminAddUser: creates account on behalf of
        another
- editUsername: change login handle
- editPassword: change password
- adminDeleteUser: admin removes a user
        account
Invariants:
- email must be unique across all users
- username must be unique across all users
Integrity notes: User spans both identity
        concerns and collaboration concerns.
        This is the standard design for this
        class of app.

```

Figure 1. User Concept Example

language specification, the “following” field is always false for unauthenticated users. Once the user has looked over the properties, the agent translates them into LTL sentences when possible, here it would look like

$$G(\neg \text{authenticated} \rightarrow \neg \text{following}).$$

We note that to users, it may be easier to phrase this property as “a user must be logged in to follow another,” and write the formalized version as

$$G(\text{following} \rightarrow \text{authenticated}).$$

Additional work is needed to improve user experience when validating specifications and improve the LLM’s generation of simpler specifications.

Using these translated sentences, the agent generates PBTs using Bombadil [3] which can be run to check the properties for failures. Bombadil is a PBT framework that treats web pages as reactive systems and defining invariants and temporal properties with an LTL-based specification language. Figure 2 is the generated Bombadil test for the previous property. It uses the appearance of the follow button to determine whether the property is violated or not, i.e., that a user that is not logged in is following another user.

```
// Profile S1 (formalized): following is
// always false for unauthenticated viewers
// Observable: when unauthenticated on a
// profile page, follow button label must
// not say "Unfollow" (which would indicate
// following=true).
export const
  profile_S1_unauthenticatedFollowingFalse
  = always(() => {
    if (!isAuthenticated.current &&
        isOnProfilePage.current) {
      const label = followButtonLabel.current;
      if (label !== null) {
        return !label.startsWith("Unfollow");
      }
    }
    return true;
  });
```

Figure 2. A Bombadil Test

3 Preliminary Results

We have tested our tool on two open source applications which have been used to evaluate other automated GUI testing tools [13]. The first application is RealWorld [5], a clone of the blogging site Medium created as an example application that uses the Angular frontend. The second is 4ga Boards [1], a task tracking application. For the RealWorld app, our tool identified 14 concepts, nine dependencies between concepts, and two synchronizations. For example, the article concept depends on the user concept since every article is authored by a particular user, and the synchronization between the follow concept and the article concept reflects that the feed is entirely determined by who the user follows. In the second phase, our tool identified 94 specifications based on the concept analysis, over half of which focused on authentication and authorization.

Our tool identified 23 concepts for the 4ga Boards app, primarily organized around the hierarchy of Kanban boards: cards exist in a list which lives in a board for a particular project. It also generated 161 specifications, with 40% related to authentication and authorization. The other specifications focused on entity-relationship requirements (e.g., every card belongs to exactly one board). This resulted in very few specs that were fully formalized as LTL. To assess the accuracy of the specifications, we sampled five of the specifications and judged whether they matched our expectations of the site's behavior. The tool was also able to generate Bombadil tests; however, we were not able to find any existing bugs in those applications. We plan to evaluate the quality of the tests by seeding bugs in the source code via mutation testing.

4 Related Work

There are many existing tools for conducting automated end to end web application testing using LLMs. GenIA-E2ETest writes example-based E2E tests based on natural language unstructured test scenarios. In contrast, our tool does not rely on existing test scenarios or given natural language descriptions of the web application. Others, such as Temac [13] use LLMs to explore the entirety of a web application. While we also use agents to explore an application, the agents only explore a subset of the application by design, as running these agents can be expensive. Instead, we rely on the concept analysis to infer the existence of particular states. If these inferred states do not exist, the user may choose to consider that to be a bug, depending on the validity of the spec. AutoE2E is the tool more similar to SPINACH, and uses LLMs to infer the existence of specific features. The tool then writes tests for each of the inferred features. AutoE2E uses action traces to infer features and writes example-based tests with a one-to-one mapping of tests to features. This is similar to software concepts' focus on system behavior, but our tool uses property-based testing to check a wider range of inputs.

5 Future Work and Conclusion

We see two main avenues for future work. The first is improving the execution of the property-based tests. Often, the specifications generated by our tool had complex preconditions. For instance, the specification that every article created must have at least one tag associated with it requires the actions of logging in and then creating an article, which in turn require the individual actions of clicking the appropriate text entry boxes and buttons in the correct order. However, generating this sequence of actions can be rare, resulting in the specification being tested infrequently. In our preliminary results, many properties were never adequately checked because the actions generated never put the application in the necessary states. Currently, Bombadil selects actions from a weighted tree of possible interactions. While this can allow broad exploration of the application, it can take a long time to reach more error-prone states. Better generators of action traces can more robustly exercise the properties written by our system in less time. There has been recently published work in this direction that allows for test generators that produce constrained sequences of actions [17] and can be incorporated in a new web app testing PBT tool.

We also hope to study whether LTL is the appropriate language for representing properties of web applications. While our tool allows users to inspect the inferred properties in natural language, the user still needs to ensure that the LLM-translated LTL matches the natural language. There are ways to increase assurance in these autoformalizations [11], but fundamentally the user must have an understanding of

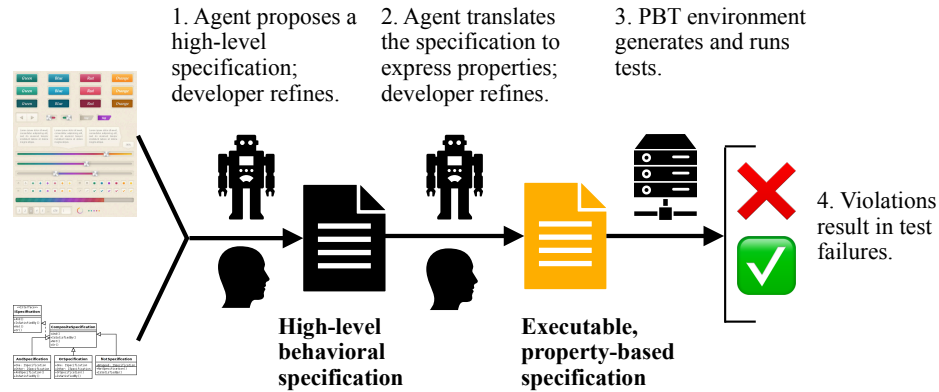


Figure 3. Overview of SPINACH.

LTL, which can be difficult to understand [8]. If most specifications for web applications can be written with a logic that's easier for users to understand, then this can reduce the burden of manual verification of LLM output. In addition, many of the specifications produced by our tool so far are not suited for LTL formalization. These specifications rely on counts, uniqueness of identifiers, and entity-relationship requirements. Currently, Bombadil does not have the flexibility to handle all such specifications.

References

- [1] [n. d.]. 4ga Boards. <https://github.com/RARgames/4gaBoards>.
- [2] [n. d.]. Base44. <https://base44.com/>. Accessed: 2026-06-26.
- [3] [n. d.]. Bombadil Manual. <https://antithesishq.github.io/bombadil/index.html>. Accessed: 2026-06-26.
- [4] [n. d.]. Lovable. <http://lovable.dev>. Accessed: 2026-06-26.
- [5] [n. d.]. RealWorld. <https://github.com/realworld-apps/angular-realworld-example-app>.
- [6] Evan Caragay, Katherine Xiong, Jonathan Zong, and Daniel Jackson. 2024. Beyond Dark Patterns: A Concept-Based Framework for Ethical Software Design. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 291, 16 pages. doi:10.1145/3613904.3642781
- [7] Harrison Goldstein, Joseph W Cutler, Daniel Dickstein, Benjamin C Pierce, and Andrew Head. 2024. Property-based testing in practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [8] Ben Greenman, Sam Saarinen, Tim Nelson, and Shriram Krishnamurthi. 2022. Little tricky logic: misconceptions in the understanding of LTL. *arXiv preprint arXiv:2211.01677* (2022).
- [9] Daniel Jackson. 2021. The essence of software: why concepts matter for great design. (2021).
- [10] Lucas Jing, Xinqi Wang, Liao Zhang, and Simon S Du. 2026. PBT-Bench: Benchmarking AI Agents on Property-Based Testing. *arXiv preprint arXiv:2605.15229* (2026).
- [11] Eunsuk Kang, Sam Procter, Teresa Fortes, and Yining She. [n. d.]. Ambiguity Reduction in Natural Language Requirements through Auto-formalization. ([n. d.]).
- [12] Shuvendu K Lahirie. 2024. Evaluating llm-driven user-intent formalization for verification-aware languages. In *2024 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, 142–147.
- [13] Chenxu Liu, Zhiyu Gu, Guoquan Wu, Ying Zhang, Jun Wei, and Tao Xie. 2025. Temac: Multi-agent collaboration for automated web gui testing. *arXiv preprint arXiv:2506.00520* (2025).
- [14] Giovanni Quattrocchi, Liliana Pasquale, Paola Spoletini, and Luciano Baresi. 2026. Can LLMs Generate User Stories and Assess Their Quality? *IEEE Transactions on Software Engineering* 52, 5 (2026), 1773–1790. doi:10.1109/TSE.2026.3670612
- [15] Savitha Ravi and Michael Coblenz. 2025. An Empirical Evaluation of Property-Based Testing in Python. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 3897–3923.
- [16] Vasudev Vikram, Caroline Lemieux, Joshua Sunshine, and Rohan Padhye. 2023. Can large language models write good property-based tests? *arXiv preprint arXiv:2307.04346* (2023).
- [17] Zhe Zhou, Ankush Desai, Benjamin Delaware, and Suresh Jagannathan. 2026. Trace-Guided Synthesis of Effectful Test Generators. *Proceedings of the ACM on Programming Languages* 10, PLDI (2026), 430–454.